

Advanced Topics in Software Engineering

Credit 20

Level 7

Academic Year 2025-26

Copyright © 2024 University of West London

Permission granted to reproduce solely for the purpose of teaching and learning at the University of West London and its approved academic partners.

You are provided with study materials for your personal use only. You must not share these with others or upload them to websites. Any student who is found to have shared materials, particularly for personal gain, will be subject to disciplinary action if appropriate.

Table of contents

1. Team contact details	3
2. Module overview	3
3. Preparing for your Assessment.....	3
4. Summative Assessments.....	4
4.1 Summative Assessment 1	4
4.2 Summative Assessment 2	9
4.3 Summative Assessment 3	13
5. Learning materials.....	15
5.1 Essential Reading	15
5.1 Reference Reading	16
6. Maintaining Academic Honesty and Integrity.....	17
7. Meeting Deadlines	17
8. Getting Support	17
9. Glossary and Acronyms	17

1. Team contact details

Module Leader	Dr Cain Kazimoglu
Email	Cain.kazimoglu@uwl.ac.uk
Module Tutor / Lecturer	Dr Cain Kazimoglu
Email	Cain.kazimoglu@uwl.ac.uk
Module Admin	Milda Petrauskyte
Email	Milda.Petrauskyte@uwl.ac.uk
Subject Librarian	Edyta Krol
Email	Edyta.Krol@uwl.ac.uk
External Examiner	Dr Suraj Ajit, University of Northampton

2. Module overview

Content of the module includes:

- An Introduction to the Module, and Software Engineering.
- Software Processes and Process Models
- Agile Development and beyond (Part 1 and Part 2)
- Requirements Engineering
- System Modelling
- Software Architectural Design
- Software Design and Implementation Practices
- Implementation and Testing
- Software Development Support Tools
- Low Code & No Code Development
- Software Evolution and Maintenance
- Software Project Management

Please refer to Blackboard for detailed information on each of the weekly learning sessions:
<https://online.uwl.ac.uk>

3. Preparing for your Assessment

A key part of your learning will be preparation for your summative assessment. You will be provided feedback on your formative assessments, and this will help you to better understand what is required of you when you submit your summative assessment. Please see below guidance on your formative assessment and how to access your feedback.

This module integrates formative assessments within laboratory sessions, designed as practical exercises that reinforce the theoretical concepts covered in lectures earlier the same day. These hands-on activities provide students with an opportunity to apply their knowledge in a structured environment, fostering a deeper understanding of the subject matter. By engaging in practical problem-solving and experimentation, students receive immediate feedback on their progress, allowing for continuous improvement and skill development. This approach ensures alignment between theoretical instruction and practical application, enhancing both comprehension and retention of key concepts.

The learning outcomes of the module are listed below. Upon completing the module, students will be able to:

No.	Learning Outcome	Marking Criteria
LO1	Identify main principles, methods and techniques used in Software Engineering Industry today.	Assessment 1
LO2	Implement Software Development Life Cycle practices and apply agile design and development methods to develop full stack software.	Assessment 2
LO3	Prepare Process Requirement Engineering and Modelling Documentation.	Assessment 3
LO4	Practice System modelling tools and diagrams.	Assessment 2 & Assessment 3
LO5	Utilize System Testing stages and system maintenance methods.	Assessment 2 & Assessment 3

The above learning outcomes are measured in three different assessments. All three assessments in this module are all explained in great detail down below:

4. Summative Assessments

As mentioned above, there are three summative assessments in this module. Each assessment must be completed by students, and each student must submit their work on Turnitin via a link created by the module leader. Two of these assessments are designed to be team-work whereas the last one is an individual work.

4.1 Summative Assessment 1

Assessment title	A1 – Presenting Recent Research in Software Engineering
Submission date and time	Week 5 – check the exact date on the module page
Assessment type:	Teamwork
Word Count (or equivalent)	A presentation video done by team members
Video demo:	Minimum 6, maximum 12 minutes video demonstration. Only one and the same video is submitted by each team member. Each student in the team must speak with their own voice.
Where to submit	Turn it in
Feedback date	3 weeks (15 workdays)
Assessment Weighting	30%
PSRB requirements (if applicable)	-
Type:	Research paper presentation by a video demo
Number of students in each team:	4 or 5 students per team

[Submitting, feedback & grades online using Blackboard](#)

Assessment 1 Description: Recent Research investigation in Software Engineering

The primary aim of this assignment is to investigate and critically analyse a focused, current issue within Software Engineering, rather than to explain general concepts or provide textbook-style overviews. Students are expected to engage with ongoing developments, emerging debates, market trends, and research questions, and present evidence-based insights using an investigative, documentary-style video format. This assessment is designed to develop the ability to interrogate claims, examine real-world data, interpret academic and industry evidence, and communicate findings in a natural, engaging, and analytical manner.

Students must work in teams to select a contemporary topic within Software Engineering and produce a 6–12 minute mini-documentary or YouTube-style research video. Each team must consist of ideally 4 group members. The video **must not be a narrated PowerPoint presentation**. It should instead resemble an analytical documentary, research podcast, or investigative explainer that integrates discussion, visual evidence, and commentary.

Topic selection must be specific and focused. **Teams must avoid broad descriptive questions such as “What is AI?” or “Types of Machine Learning.” Instead, they should investigate one concrete aspect, trend, problem, or research direction within the chosen domain.**

Examples of appropriate focus:

- viability and impact of AI working memory
- evidence for or against an AI investment bubble
- security implications of low-code adoption
- limits of DevOps automation in practice

The emphasis is on ongoing developments, real data, and contested claims—**not textbook knowledge**.

Teams must produce one 6–12 minute mini-documentary or YouTube-style research video. The video must not be a recorded PowerPoint presentation. Natural discussion, visual evidence, short clips, statistics, demonstrations, and commentary are expected.

The video should feel like an analytical explainer or research podcast, not a slide narration.

Evidence expectations

Arguments must be supported using multiple sources, including:

- peer-reviewed publications (minimum 5, maximum 20)
- statistics, datasets, and usage reports
- industry or technical reports
- credible technology journalism and objective news sources
- interviews, demonstrations, or case evidence where relevant

Academic literature alone is not sufficient; evidence must reflect current developments.

Structure of the video:

1. **Topic and focus**
State the precise issue being investigated and why it matters now.
2. **Evidence overviews**
Key academic, industry, and data sources informing the investigation.
3. **Analysis**
Interpretation of data, comparisons, demonstrations, and critical discussion.
4. **Implications**
What the findings mean for Software Engineering practice or research.
5. **Outlook**
Likely developments, uncertainties, and open questions.

Team contribution and submission

Each student must investigate one aspect of the topic and contribute to a single integrated narrative. Individual presentations stitched together are not acceptable.

Only one video must be submitted per team. All members upload the same link. The video must be publicly accessible (UWL cloud, Panopto, or YouTube). Inaccessible links during marking will be penalised.

Topic areas (you need to specify an area under these categories, i.e. AI bubble being built, If working memory is the next stage in AI architecture, what would it bring and when?)

- AI-driven development tools
- DevOps and DevSecOps
- Cloud-native engineering
- Low-code/no-code platforms
- Software quality engineering
- Cybersecurity trends
- Data science in software engineering
- Blockchain systems
- Mobile and cross-platform development
- Mixed reality engineering environments
- Big data systems

Teams may choose the same broad area, but the investigative focus must be original, narrow, and evidence-driven.

Most common mistakes made in this assessment is:

- a) power point like presentation (unproductive, interesting delivery)
- b) same repeated unrelated gifs or videos used for the video (finding unrelated gifs or images)
- c) AI voice usage instead of students naturally speaking with their own voice
- d) not dividing the job into separate portions and making sure that each delivered delivered their own part.
- e) constructing the video manually instead of using existing AI video editors (you need to talk with your own voice, but video can be constructed using AI tools)
- f) reading a script
- g) not displaying the name, surname, and ID of the person talking on the screen at that moment.
- h) not telling a coherent full story divided into separate parts in the video

No.	Learning Outcome	Assessed via	
LO1	Identify the main principles, methods and techniques used in Software Engineering Industry today.	Assessment 1	
Criteria	Description	Mark	Learning Outcomes
A turn it in submission by each member for their team presentation link and their own individual responsibilities in conducting research.	Minimum 6, maximum 10 minutes video demonstration.	30% of overall mark	LO1

Assessment Criteria for A1

Criteria	Description	Mark (30)
Research Validity & Integrity	Originality of the work. Does the research go from an abstract explanation to detailed academic papers and/or stats? Did students identify the main points precisely in the research area? Is the research conducted on a specific scope?	10
Critical Analysis and Discussions	Do the students explain their research by showing statistical and rigorous data rather than anecdotal results? Do students reference academic research and/or objective resources? current literature of the work? What is known and what is in the future of this field? Where is the direction heading?	10
Individual Performance	Is the work professional and clearly presented? Is it clear who is talking and what they are saying? Is the work scripted, or do students simply transfer their knowledge? Do students engage and actually talk to an audience? Do they sound like they know what they talk about or they just read notes?	10

4.2 Summative Assessment 2

Assessment title	A2 – Agile Development, co-pilot and version control system use
Submission date and time	Week 9 – check the exact date on the module page
Assessment type:	Teamwork
What to submit:	Report + Team Video Demo (5-10 mins)
Word Count (or equivalent)	Report minimum 1500, maximum 4000 words.
Where to submit	Turn it in
Feedback date	3 weeks (15 working days)
Assessment Weighting	30%
PSRB requirements (if applicable)	-
Type:	Agile development practice with 3 different prototypes.
Number of students in each team:	4 or 5 students per team

Submitting, feedback & grades online using Blackboard

Agile Development, Management and the practice of Git and Co-pilot features.

No.	Learning Outcome	Assessed via
LO2	Implement Software Development Life Cycle practices and apply agile design and development methods to develop full stack software.	Assessment 2
LO4	Practice System modelling tools and diagrams.	Assessment 2
LO5	Utilize System Testing stages and system maintenance methods.	Assessment 2

Criteria	What to submit	Mark	Learning Outcomes
Agile Development and Management practice with 3 different prototypes.	A report written by team members – every member of each team submits the same report. If you do not submit the report, you will not be marked, and therefore, make sure to submit the report. A video demonstration of the three prototypes developed. How prototypes differ from one another. What agile development and management methods were used in the assessment? Which co-pilot and IDEs are used with which technologies? How did the members practice Git usage? What was the role of AI? What percentage of the code is AI generated?	30% of overall mark	LO 2, 4, 5

Assessment 2 Description: Agile Development and Management

This assignment is designed to simulate how modern software teams actually build and release products using agile development practices supported by GitHub, testing, and automated build and delivery processes. Students will work in teams to develop a small web or mobile prototype over weekly cycles, producing three evolving versions of the same software. The purpose is not to deliver a polished commercial application but to demonstrate how real agile work happens: planning, iteration, version control, testing, collaboration, and continuous improvement.

Each team must operate using Scrum, Kanban, or a combination of both. Work must be organised through a backlog, tasks must be pulled and completed collaboratively, and each member must take a defined role which are Scrum Master/Agile Coach, UX Lead, Automation Lead and Developers. Across three sprint cycles, the team must plan work, implement features, test them, review progress, and release a new version.

GitHub must be used as the central working environment. Students should not treat it as a file storage platform but as a development workspace. The project must use feature branches, pull requests, code review, and visible commit history from all team members. Changes should appear gradually as the software evolves from version to version, showing collaboration and shared ownership. The development process must be observable through issues, task tracking, and version history.

Testing must be built into development rather than added at the end. Students are expected to write simple unit tests and improve them as features evolve. This follows the logic of test-driven development: building small pieces, testing them, and refining them. The goal is not perfection but demonstrating that the team understands how testing shapes development decisions.

Automation must also be introduced. When new code is uploaded, it should automatically build and run tests, and if successful, produce a deployable version or preview. This can be done using free tools. For example, GitHub Actions can automatically run tests whenever code is pushed. A simple deployment can be created using Netlify or Vercel for web apps, or Firebase hosting for mobile/web prototypes. The intention is to show how development connects directly to delivery without manual packaging each time.

The DevOps element does not require complex infrastructure. A realistic setup could be: GitHub repository for collaboration, GitHub Actions for automated build and testing, and Netlify for deployment of the prototype website. This is free, simple, and reflects real workflows used in industry. The key is to demonstrate that code moves from development to testing to delivery in a structured and repeatable way.

At the end of the assignment, teams must show how their product evolved across three versions and how the process supported that evolution. The deliverables must make it clear how work was planned, how features were built and tested, how GitHub supported collaboration, and how automation supported development. The aim is to demonstrate understanding of agile practice as a working system, not as a theory.

An example:

A team might build a small Spotify-like music discovery app. Version 1 could allow users to browse a simple list of songs and play previews. Version 2 might introduce playlists and user interaction. Version 3 could add recommendations or data visualisation of listening behaviour. Each version should reflect planning, development, testing, and improvement.

In this example, the backlog might contain tasks such as “build audio player,” “create playlist feature,” or “test search function.” Team members pull tasks, implement them on separate branches, and submit pull requests. Tests might check whether the playlist feature saves correctly or whether the search returns expected results. Every time code is merged, GitHub Actions runs those tests automatically. If they pass, the app is automatically deployed to Netlify so the team can see the latest working version online.

Role definition for the assessment:

Notice that each role is involved in some kind of coding. The goal of the assessment is not to build a polished beautiful and perfectly working application. The goal is to engage with software engineering practices aligned with agile development and providing evidence for each role.

Scrum Master / Agile Coach

- *Contribute code each sprint* (feature, refactor, or tooling) while facilitating planning, reviews, and retrospectives.
- *Implement and maintain workflow-supporting artefacts in code* where relevant (e.g., templates, scripts, documentation automation, configuration files).
- Remove technical blockers by assisting with integration issues, merge conflicts, or build problems in the codebase.

UX / Product Lead

- Translate user stories into *implementable components and contribute front-end/interface code or interaction logic*.
- Encode acceptance criteria into test cases, validation logic, or UI behaviour within the application.
- Prototype and refine user flows through working code rather than static mock-ups.

Automation / DevOps Lead

- Write and maintain CI/CD configuration code (e.g., GitHub Actions workflows, deployment scripts, environment setup).
- *Implement build scripts, test runners, and integration checks that execute automatically*.
- Contribute to application code when integrating deployment requirements, APIs, or runtime configurations.

Front-end / Back-end Developers

- *Implement core application* features and supporting logic across sprint cycles.
- *Write and maintain unit tests* following test-driven development practices.
- Perform code reviews, refactoring, and collaborative contributions through branching and pull requests.

Assessment Criteria for A2

Criteria	Description	Mark (30)
Prototypes	Evaluation of prototypes, how well each version differs from one another. Is there evidence that a realistic agile development approach was taken place? What is the quality of the work? Did the team members all work in the prototypes or is there uneven work among the team members? The more evenly the students are involved in coding, the more evenly the marks will be distributed. Students who did not code in prototypes will not get a mark in this category.	10
Git features and Co-pilot usage	How well the team used Git-features. This should be discussed both in the Report and in the Video. The Video discussion should be upfront and provide evidence, whereas the report should go into more detail. Those students who did not use Git features or provided any evidence regarding the co-pilot usage will not get a mark in this category.	10
Professionalism	How professional the work is presented both in the video and in the report. Did all members talk in the video demonstration? Is the video clearly audible and all features of prototypes are demonstrated? Is there any evidence of teamwork management in agile development? Are the Responsibilities broken down evenly among the team members? Did everyone talk and explained their part without a script? Did they engage with the audience genuinely?	10

4.3 Summative Assessment 3

Assessment title	A3 – Plan driven development
Submission date and time	Week 13– check the exact date on the module page
Assessment type:	Individual
Word Count (or equivalent)	Report minimum 1500, maximum 3000 words.
Video demo	Minimum 5, maximum 10 minutes.
Where to submit	Turn it in
Feedback date	3 weeks (15 working days)
Assessment Weighting	30%
PSRB requirements (if applicable)	-
Type:	A plan driven development approach with a prototype should be presented. Additionally, a video demonstration showing the features of the prototype must be done. Initial coding is necessary for this assessment (e.g., User Interface, basic interactions), however a full implementation is optional and will be considered as going above and beyond.

Submitting, feedback & grades online using Blackboard

Plan Driven Development Approach

Learning Outcome	Assessed via
Prepare Process Requirement Engineering and Modelling Documentation.	Assessment 3
Practice System modelling tools and diagrams.	Assessment 3
Utilize System Testing stages and system maintenance methods.	Assessment 3

Criteria	What to submit	Mark	Learning Outcomes
Plan driven approach with basic implementation	A report written by a student based on a plan driven approach model. A video demonstration will be submitted for the implementation. The implementation does not need to be complete.	40% of overall mark	LO 3, 4, 5

Assessment 3 Description: Plan driven development

This is the last and the individual assessment in the module which is a plan-driven software engineering assessment focused on predictability, stability of requirements, and design-before-build practice. Unlike earlier agile work, the objective is to define requirements early, model the system rigorously, and implement a controlled portion of it with structured testing and traceability.

Students will design a performance-oriented movie theatre seating allocation system. The system must allocate seats based on defined constraints such as group size, adjacency, availability, and allocation efficiency. The emphasis is on defining and stabilising rules in advance and engineering a solution that behaves consistently.

Work must follow a structured flow: requirements → modelling → implementation → testing. Requirements must be established before development and must guide all design and implementation decisions.

Students are required to produce the following:

1. Requirements

At least 3 non-functional requirements, each linked to a system quality goal (e.g., performance, reliability, consistency).

At least 8 functional requirements describing what the seating system must do.

2. UML modelling (completed before implementation)

1 use-case diagram representing system functionality and user interaction.

At least 3 sequence diagrams showing interactions between system components during booking and allocation (this will focus on booking algorithm, not on registration, sign up or other features).

At least 3 class diagrams defining system structure, variables, objects, methods, and relationships.

3. Algorithm design

1 activity diagram or flowchart specifically modelling the seating allocation algorithm, including decision logic, constraints, allocation paths, and exception handling.

4. Implementation

Partial implementation demonstrating the core seating allocation logic and essential system behaviour. Any language can be used for this but javascript or python is strongly suggested.

5. Testing

Evidence of unit testing of the seating algorithm.

Evidence of integration and acceptance testing derived from requirements.

6. Traceability

Clear mappings linking requirements to UML models, algorithm design, implementation, and test cases.

7. Reflection

Critical discussion of planning, modelling, testing, implementation decisions, and limitations of the solution.

The assessment evaluates the ability to engineer a predictable system where behaviour is defined in advance, modelled clearly, implemented in a controlled way, and validated through structured testing.

Criteria	Description	Mark (40)
The Plan driven approach	This includes test-driven development, diagrams, and the pre- development design plan.	20
Implementation and Software Development Management	This includes implementation, the quality of the work done, the programming code evaluation, unit and acceptance testing, and students' reflection. Implementation does not need to be complete, however if students complete a full implementation, they will be perceived to go above and beyond.	20

5. Learning materials

The reading list for this module is available on Blackboard in the module area and online by searching [reading lists](#). This shows real-time availability of books in the library and provides direct links to digital items, recommended by your lecturer.

Remember to log into Blackboard daily to receive all the latest news and support available at your module information sites!

[Subject guides](#) are also available to help you find relevant information for assignments, with contact details of the [Subject Librarian](#) for your School.

5.1 Essential Reading

Modern Software Engineering: Doing What Works to Build Better Software Faster, David Farley, Addison-Wesley Professional, 2021, ISBN 0137314914.

Software Engineering (10th Edition), Ian Sommerville, Published by Pearson, 2016, ISBN-10: 1292096136, ISBN-13: 9781292096131.

5.1 Reference Reading

Agile Software Development: Principles, Patterns, and Practices, Robert C. Martin, Published by Pearson, 2013, ISBN-10: 1292025948, ISBN-13: 9781292025940.

Head first Design Patterns, Eric Freeman, Freeman Elisabeth, Kathy Sierra, and Bert Bates (2004), Published by O'Reilly, 2004, ISBN-10: 0596007124, ISBN-13: 9780596007126.

6. Maintaining Academic Honesty and Integrity

Academic Integrity means avoiding plagiarism and cheating and owning your own work, the use of essay mills and AI content is also considered academic misconduct. This is when you submit a piece of work which is not completely your own, but which you are presenting as your own without acknowledging the author or properly referencing the original source. All your work must demonstrate Academic Integrity; it must be an honest and fair submission, complying with all the requirements of the assessment. Failure to meet these standards of behaviour and practice is academic misconduct, which can result in penalties being applied under the [Academic Offences Regulations](#). You can get support with your academic writing by speaking to our [Study support](#) team.

7. Meeting Deadlines

You should always try your best to submit your work on time. If your circumstances mean that you are not able to submit on time or are unable to attend an in-person assessment like an exam or in-class test, then you can request [Exceptional circumstances](#) for the assessment. An **extension** allows you to submit coursework up to 10 calendar days late without penalty, (calendar days include all weekends and bank holidays where the University is open). Without an extension, the maximum mark you will be able to get for that work will be the pass mark. **Mitigation** allows you a further attempt without penalty if you fail an assessment or do not submit.

You can apply for an extension or mitigation by self-certifying that you have exceptional circumstances which affected your ability to undertake the assessment. **Self-certifying** means that evidence does not have to be provided, although the University reserves the right to request evidence. All self-certified requests must be made before the deadline and detail the exceptional circumstances that have prevented you from submitting by the original submission deadline. **You can only self-certify three assessments per academic year.** If you have used all your self-certification opportunities, or requested mitigation after the deadline, you will need to provide evidence of your [exceptional circumstances](#) for your request to be granted.

8. Getting Support

There may be times when you experience circumstances outside of your control and talking to your Module Leader/Tutor and seeking help from other support services in the university will keep you on track with your studies. You can access support and guidance and confidential help at our [Support for current students](#) page and by visiting the [Student Hub](#). It is equally important to consider your safety and the safety of others around you, especially [how to stay safe online and ensure your communications are secure and appropriate](#). You should also familiarise yourself with the University [Safeguarding Policy, process, and procedure](#).

Your [Students' Union Advice Team](#) can help you to obtain the right support.

9. Glossary and Acronyms

Agile Development – A software development methodology that emphasizes iterative development, collaboration, and adaptability.

Artificial Intelligence (AI) – The simulation of human intelligence processes by machines, particularly computer systems.

Blackboard – A learning management system (LMS) used for online submissions, grading, and feedback.

Cloud Computing – The practice of using a network of remote servers hosted on the internet to store, manage, and process data.

Co-Pilot – An AI-powered coding assistant that helps developers by suggesting code snippets and automating repetitive tasks.

Critical Analysis – The systematic examination of an issue or research topic, including evaluation of claims, evidence, and counterarguments.

DevOps – A set of practices that combines software development (Dev) and IT operations (Ops) to improve efficiency and continuous delivery.

DevSecOps – A security-focused extension of DevOps that integrates security practices into the software development lifecycle.

Documentation – Written text or illustrations accompanying software or hardware that explain how it operates.

Git – A distributed version control system used for tracking changes in source code during software development.

GitHub – A web-based platform for version control and collaboration, allowing developers to manage Git repositories.

IDE (Integrated Development Environment) – A software application that provides comprehensive facilities for software development, including a code editor, debugger, and compiler.

Interactive Video – A multimedia format where viewers can interact with video elements for an engaging learning experience.

Kanban Board – A visual tool used in Agile development to track and manage workflow and tasks.

Machine Learning (ML) – A subset of AI that enables computers to learn from data and improve performance on tasks over time.

Methodology – A structured framework that guides research or software development processes.

Non-Functional Requirements (NFRs) – System attributes such as security, performance, maintainability, and usability that define the quality of the software.

Plan-Driven Development – A software engineering approach that relies on detailed upfront planning and documentation before implementation.

Prototype – A preliminary version of a product or system used to test and refine concepts.

Requirements Engineering – The process of defining, documenting, and maintaining software requirements.

Software Development Life Cycle (SDLC) – The process followed for software development, from planning to maintenance.

System Modelling – The use of diagrams and models to represent and analyse system components and interactions.

Test-Driven Development (TDD) – A software development practice where tests are written before the code is implemented.

Turnitin – A plagiarism detection and grading tool used for academic submissions.

User Interface (UI) – The graphical layout and interaction components of a software application.

Version Control System (VCS) – A system that records changes to a file or set of files over time so that previous versions can be recalled when needed.

Video Demonstration – A recorded presentation showcasing software functionality, research findings, or project outcomes.